

Память, Указатели, Строки и Файлы в C++

Часть 1: Области памяти и жизненный цикл переменных

1.1 Три основные области памяти

```
#include <iostream>
using namespace std;

// Глобальная переменная - статическая память
int globalVar = 100; // Существует всю программу

void demonstrateMemoryAreas() {
    cout << "=== ОБЛАСТИ ПАМЯТИ ===" << endl;

    // 1. Стек (Stack) - автоматическое управление
    cout << "\n1. СТЕК (локальные переменные):" << endl;
    cout << "-----" << endl;

    int stackVar = 10; // Переменная в стеке
    cout << "stackVar = " << stackVar << endl;
    cout << "Адрес stackVar: " << &stackVar << endl;

    // Особенности стека:
    // - Автоматическое выделение/освобождение
    // - Быстрый доступ
    // - Ограниченный размер (~1-8 МБ в зависимости от ОС)
    // - LIFO (Last In, First Out)

    // 2. Куча (Heap) - динамическое управление
    cout << "\n2. КУЧА (динамическая память):" << endl;
    cout << "-----" << endl;
```

```

int* heapVar = new int(20); // Выделяем память в куче
cout << "*heapVar = " << *heapVar << endl;
cout << "Адрес heapVar: " << heapVar << endl;
cout << "Адрес указателя &heapVar: " << &heapVar << endl;

// Особенности кучи:
// - Ручное управление (new/delete)
// - Большой размер (ограничен ОЗУ)
// - Медленнее стека
// - Память доступна до явного освобождения

delete heapVar; // ОБЯЗАТЕЛЬНО освобождаем память!

// 3. Статическая память
cout << "\n3. СТАТИЧЕСКАЯ ПАМЯТЬ:" << endl;
cout << "-----" << endl;

static int staticVar = 30; // Статическая локальная переменная
cout << "globalVar = " << globalVar << endl;
cout << "staticVar = " << staticVar << endl;
cout << "Адрес globalVar: " << &globalVar << endl;
cout << "Адрес staticVar: " << &staticVar << endl;

// Особенности статической памяти:
// - Существует всю программу
// - Инициализируется один раз
// - Для глобальных и static переменных
}

// Демонстрация жизненного цикла
void lifecycleDemo() {
    cout << "\n\n=== ЖИЗНЕННЫЙ ЦИКЛ ПЕРЕМЕННЫХ ===" << endl;
    cout << "===== " << endl;

    // 1. Автоматические переменные (стек)
    {
        cout << "\n1. Блок 1 начался" << endl;
        int autoVar = 42;
        cout << "autoVar создана: " << autoVar << endl;
        cout << "autoVar уничтожится при выходе из блока" << endl;
    } // Здесь autoVar уничтожается
}

```

```
cout << "Блок 1 завершился - autoVar уничтожена" << endl;

// 2. Статические переменные
cout << "\n2. Статические переменные:" << endl;
for (int i = 0; i < 3; i++) {
    static int counter = 0; // Инициализируется один раз
    int temp = 0;           // Создается заново каждый раз

    counter++;
    temp++;

    cout << "Итерация " << i << ": counter = " << counter
    cout << ", temp = " << temp << endl;
}

// 3. Динамические переменные (куча)
cout << "\n3. Динамические переменные:" << endl;
int* dynamicVar = new int(777);
cout << "Выделена память в куче: " << *dynamicVar << endl

// Память НЕ освобождается автоматически!
// Если не сделать delete, будет утечка памяти

delete dynamicVar; // Освобождаем память
cout << "Память освобождена (delete)" << endl;
}

int main() {
    demonstrateMemoryAreas();
    lifecycleDemo();

    // Дополнительная демонстрация размеров
    cout << "\n\n=== РАЗМЕРЫ ПАМЯТИ ===" << endl;
    cout << "======" << endl;

    // Попытка выделить слишком много в стеке
    try {
        // int hugeArray[100000000]; // ~40MB - СЛИШКОМ МНОГО
        // Это вызовет Stack Overflow
    } catch (...) {
        cout << "Слишком большой массив для стека!" << endl;
    }
}
```

```

// А в куче можно
int* hugeArray = new int[100000000]; // 40MB в куче - норм
cout << "Выделено 40MB в куче успешно" << endl;
delete[] hugeArray;

return 0;
}

```

1.2 Наглядная аналогия

Представьте себе:

- **Стек** как стопку тарелок в столовой
- **Кучу** как склад с полками
- **Статическую память** как постоянную витрину

Часть 2: Указатели и динамическая память

```

#include <iostream>
#include <cstring> // для memset, memcpy
using namespace std;

void pointersBasics() {
    cout << "=== ОСНОВЫ УКАЗАТЕЛЕЙ ===" << endl;
    cout << "===== " << endl;

    int x = 10;
    int y = 20;

    // 1. Объявление и инициализация указателей
    cout << "\n1. Базовые указатели:" << endl;

    int* ptr = &x; // ptr указывает на x
    cout << "x = " << x << endl;
    cout << "Адрес x (&x) = " << &x << endl;
    cout << "ptr = " << ptr << endl;
    cout << "*ptr = " << *ptr << endl; // разыменование

    // 2. Изменение значения через указатель

```

```

cout << "\n2. Изменение через указатель:" << endl;
*ptr = 15; // меняем x через указатель
cout << "После *ptr = 15:" << endl;
cout << "x = " << x << endl;
cout << "*ptr = " << *ptr << endl;

// 3. Перепривязка указателя
cout << "\n3. Перепривязка указателя:" << endl;
ptr = &y; // теперь ptr указывает на y
cout << "После ptr = &y:" << endl;
cout << "y = " << y << endl;
cout << "*ptr = " << *ptr << endl;

// 4. Указатель на указатель
cout << "\n4. Указатель на указатель:" << endl;
int** ptr_to_ptr = &ptr;
cout << "ptr = " << ptr << endl;
cout << "*ptr = " << *ptr << endl;
cout << "ptr_to_ptr = " << ptr_to_ptr << endl;
cout << "*ptr_to_ptr = " << *ptr_to_ptr << endl;
cout << "**ptr_to_ptr = " << **ptr_to_ptr << endl;
}

void dynamicMemory() {
    cout << "\n\n=== ДИНАМИЧЕСКАЯ ПАМЯТЬ ===" << endl;
    cout << "===== " << endl;

    // 1. Выделение памяти для одного значения
    cout << "\n1. Одиночное выделение:" << endl;

    int* single = new int(42); // выделяем и инициализируем
    cout << "*single = " << *single << endl;

    // Меняем значение
    *single = 100;
    cout << "После изменения: *single = " << *single << endl;

    delete single; // ОБЯЗАТЕЛЬНО освободить!
    single = nullptr; // Хорошая практика

    // 2. Выделение памяти для массива
    cout << "\n2. Динамические массивы:" << endl;

```

```
int size = 5;
int* array = new int[size]; // массив из 5 int

// Инициализируем массив
for (int i = 0; i < size; i++) {
    array[i] = i * 10;
    cout << "array[" << i << "] = " << array[i] << endl;
}

// Альтернативный доступ через указательную арифметику
cout << "\nДоступ через указательную арифметику:" << endl
for (int i = 0; i < size; i++) {
    cout << "*(array + " << i << ") = " << *(array + i) << endl;
}

delete[] array; // ОБЯЗАТЕЛЬНО с [] для массивов!
array = nullptr;

// 3. Двумерный динамический массив
cout << "\n3. Двумерный массив:" << endl;

int rows = 3, cols = 4;

// Создаем массив указателей на строки
int** matrix = new int*[rows];

// Для каждой строки создаем массив столбцов
for (int i = 0; i < rows; i++) {
    matrix[i] = new int[cols];

    // Заполняем значениями
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = i * 10 + j;
    }
}

// Выводим матрицу
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        cout << matrix[i][j] << " ";
    }
    cout << endl;
}
```

```

// ОЧЕНЬ ВАЖНО: освобождаем в обратном порядке!
for (int i = 0; i < rows; i++) {
    delete[] matrix[i]; // удаляем каждую строку
}
delete[] matrix; // удаляем массив указателей

// 4. Опасности динамической памяти
cout << "\n4. ОПАСНОСТИ (что НЕ делать):" << endl;

// Утечка памяти (memory leak)
int* leak = new int(5);
// Забыли delete leak - память никогда не освободится!

// Двойное удаление (double free)
int* danger = new int(10);
delete danger;
// delete danger; // ОШИБКА: повторное удаление!

// Использование после удаления (use after free)
int* zombie = new int(20);
delete zombie;
// *zombie = 30; // ОШИБКА: обращение к освобожденной па

// Неинициализированный указатель
int* wild;
// *wild = 5; // ОШИБКА: указатель указывает "в никуда"

// Решение: всегда инициализируйте nullptr
int* safe = nullptr;
if (safe != nullptr) {
    *safe = 5; // Проверили - безопасно
}
}

void pointerArithmetic() {
    cout << "\n\n=== УКАЗАТЕЛЬНАЯ АРИФМЕТИКА ===" << endl;
    cout << "===== " << endl;

    int arr[] = {10, 20, 30, 40, 50};
    int* ptr = arr; // ptr указывает на первый элемент

    cout << "Массив: ";

```

```

    for (int i = 0; i < 5; i++) {
        cout << arr[i] << " ";
    }
    cout << endl << endl;

    cout << "ptr = " << ptr << " (адрес arr[0])" << endl;
    cout << "*ptr = " << *ptr << " (значение arr[0])" << endl;

    // Инкремент указателя
    ptr++; // перемещаемся на следующий элемент
    cout << "\nПосле ptr++:" << endl;
    cout << "ptr = " << ptr << " (адрес arr[1])" << endl;
    cout << "*ptr = " << *ptr << " (значение arr[1])" << endl;

    // Разность указателей
    int* ptr2 = &arr[4];
    cout << "\nptr2 = &arr[4]" << endl;
    cout << "Разность ptr2 - ptr = " << (ptr2 - ptr) << " эле

    // Доступ через арифметику
    cout << "\nДоступ через арифметику:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "*(arr + " << i << ") = " << *(arr + i) << eni
    }
}

int main() {
    pointersBasics();
    dynamicMemory();
    pointerArithmetic();

    return 0;
}

```

Часть 3: Строки в C++

```

#include <iostream>
#include <cstring>    // для C-строк
#include <string>      // для std::string
#include <algorithm>   // для алгоритмов
using namespace std;

```

```
void cStrings() {
    cout << "=== C-СТРОКИ (массивы char) ===" << endl;
    cout << "===== " << endl;

    // 1. Разные способы создания C-строк
    cout << "\n1. Создание C-строк:" << endl;

    // Способ 1: Массив с явным размером
    char str1[20] = "Hello";
    cout << "str1: " << str1 << endl;
    cout << "Длина: " << strlen(str1) << endl;
    cout << "Размер массива: " << sizeof(str1) << endl;

    // Способ 2: Без указания размера (компилятор вычисляет)
    char str2[] = "World";
    cout << "\nstr2: " << str2 << endl;
    cout << "Размер: " << sizeof(str2) << " (включая '\\0')" .

    // Способ 3: Посимвольно
    char str3[6];
    str3[0] = 'H'; str3[1] = 'e'; str3[2] = 'l';
    str3[3] = 'l'; str3[4] = 'o'; str3[5] = '\\0';
    cout << "\nstr3: " << str3 << endl;

    // 2. Операции со строками
    cout << "\n2. Операции со строками:" << endl;

    // Копирование
    char copy[20];
    strcpy(copy, str1);
    cout << "После strcpy: " << copy << endl;

    // Конкатенация
    strcat(copy, " ");
    strcat(copy, str2);
    cout << "После strcat: " << copy << endl;

    // Сравнение
    cout << "\nСравнение строк:" << endl;
    cout << "strcmp(\"abc\", \"abc\") = " << strcmp("abc", "al
    cout << "strcmp(\"abc\", \"abd\") = " << strcmp("abc", "al
    cout << "strcmp(\"abd\", \"abc\") = " << strcmp("abd", "al
```

```

cout << "(0 - равны, <0 - первая меньше, >0 - первая болы

// 3. Опасности C-строк
cout << "\n3. ОПАСНОСТИ C-строк:" << endl;

// Переполнение буфера
char small[5];
// strcpy(small, "Very long string"); // ПЕРЕПОЛНЕНИЕ!

// Отсутствие завершающего нуля
char danger[3] = {'a', 'b', 'c'}; // Нет '\0'!
// cout << danger << endl; // Будет читать до случайного

// Решение: использовать безопасные функции
strncpy(small, "Hello", sizeof(small) - 1);
small[sizeof(small) - 1] = '\0'; // Гарантируем завершени
cout << "Безопасная копия: " << small << endl;
}

void cppStrings() {
    cout << "\n\n=== std::string (C++ строки) ===" << endl;
    cout << "===== " << endl;

    // 1. Создание и инициализация
    cout << "\n1. Создание строк:" << endl;

    string s1 = "Hello";           // из C-строки
    string s2("World");           // конструктор
    string s3(5, 'A');             // 5 символов 'A'
    string s4 = s1 + " " + s2;     // конкатенация

    cout << "s1: " << s1 << endl;
    cout << "s2: " << s2 << endl;
    cout << "s3: " << s3 << endl;
    cout << "s4: " << s4 << endl;

    // 2. Основные операции
    cout << "\n2. Операции:" << endl;

    // Длина
    cout << "Длина s4: " << s4.length() << endl;
    cout << "Пуста ли строка: " << (s4.empty() ? "да" : "нет"

```

```
// Доступ к символам
cout << "s4[0]: " << s4[0] << endl;
cout << "s4.at(6): " << s4.at(6) << endl; // с проверкой

// Изменение
s4.append("!!!");
cout << "После append: " << s4 << endl;

// Вставка
s4.insert(5, " beautiful");
cout << "После insert: " << s4 << endl;

// Замена
s4.replace(0, 5, "Hi");
cout << "После replace: " << s4 << endl;

// Удаление
s4.erase(2, 10); // удаляем 10 символов начиная с позиции
cout << "После erase: " << s4 << endl;

// 3. Поиск
cout << "\n3. Поиск:" << endl;

string text = "The quick brown fox jumps over the lazy dog";
string word = "fox";

size_t pos = text.find(word);
if (pos != string::npos) {
    cout << "Найдено '" << word << "' на позиции " << pos
} else {
    cout << "Не найдено" << endl;
}

// Поиск с конца
pos = text.rfind("the");
cout << "Последнее 'the' на позиции: " << pos << endl;

// 4. Подстроки
cout << "\n4. Подстроки:" << endl;

string substring = text.substr(10, 10); // с позиции 10,
cout << "Подстрока: " << substring << endl;
```

```
// 5. Преобразования
cout << "\n5. Преобразования:" << endl;

// string → C-string
const char* cstr = s4.c_str();
cout << "C-строка: " << cstr << endl;

// Число → string
int num = 123;
string numStr = to_string(num);
cout << "Число как строка: " << numStr << endl;

// string → число
string ageStr = "25";
int age = stoi(ageStr);
cout << "Строка как число: " << age << endl;

// 6. Итерация по строке
cout << "\n6. Итерация:" << endl;

cout << "По символам: ";
for (char c : s4) {
    cout << c << " ";
}
cout << endl;

cout << "С индексами: ";
for (size_t i = 0; i < s4.length(); i++) {
    cout << s4[i] << " ";
}
cout << endl;

// 7. Алгоритмы
cout << "\n7. Алгоритмы:" << endl;

string data = "Hello World";

// Преобразование регистра
transform(data.begin(), data.end(), data.begin(), ::toupper);
cout << "Верхний регистр: " << data << endl;

transform(data.begin(), data.end(), data.begin(), ::tolower);
cout << "Нижний регистр: " << data << endl;
```

```

    // Сортировка
    string sorted = "dcba";
    sort(sorted.begin(), sorted.end());
    cout << "Отсортировано: " << sorted << endl;
}

void stringComparison() {
    cout << "\n\n=== СРАВНЕНИЕ C-строк и std::string ===" << endl;
    cout << "===== " << endl;

    cout << "C-строки:" << endl;
    cout << "+ Быстрые для простых операций" << endl;
    cout << "+ Минимальный оверхед" << endl;
    cout << "- Опасны (переполнение буфера)" << endl;
    cout << "- Ручное управление памятью" << endl;
    cout << "- Нет встроенных методов" << endl;

    cout << "\nstd::string:" << endl;
    cout << "+ Безопасны" << endl;
    cout << "+ Автоматическое управление памятью" << endl;
    cout << "+ Много удобных методов" << endl;
    cout << "+ Легко расширяются" << endl;
    cout << "- Большой оверхед" << endl;
    cout << "- Немного медленнее для простых операций" << endl;

    cout << "\nРекомендация: всегда используйте std::string,"
    cout << "если нет строгих требований к производительности"
}

int main() {
    cStrings();
    cppStrings();
    stringComparison();

    return 0;
}

```

Часть 4: Работа с файлами

```
#include <iostream>
#include <fstream>    // файловые потоки
#include <string>
#include <vector>
#include <iomanip>     // манипуляторы вывода
#include <sstream>     // строковые потоки
using namespace std;

void textFiles() {
    cout << "=== РАБОТА С ТЕКСТОВЫМИ ФАЙЛАМИ ===" << endl;
    cout << "===== " << endl;

    // 1. Запись в файл
    cout << "\n1. Запись в файл:" << endl;

    ofstream outFile("example.txt");

    if (!outFile) {
        cerr << "Ошибка открытия файла для записи!" << endl;
        return;
    }

    // Записываем данные
    outFile << "Привет, мир!" << endl;
    outFile << "Это пример записи в файл." << endl;
    outFile << "Число: " << 42 << endl;
    outFile << "Дробное число: " << 3.14159 << endl;

    // Форматированный вывод
    outFile << fixed << setprecision(2);
    outFile << "Форматированное число: " << 3.14159 << endl;

    outFile.close();
    cout << "Данные записаны в example.txt" << endl;

    // 2. Чтение из файла
    cout << "\n2. Чтение из файла:" << endl;

    ifstream inFile("example.txt");

    if (!inFile) {
        cerr << "Ошибка открытия файла для чтения!" << endl;
```

```
        return;
    }

    string line;
    int lineNumber = 1;

    cout << "Содержимое файла:" << endl;
    cout << "-----" << endl;

    // Чтение построчно
    while (getline(inFile, line)) {
        cout << setw(2) << lineNumber++ << ": " << line << endl;
    }

    inFile.close();

    // 3. Чтение разными способами
    cout << "\n3. Разные способы чтения:" << endl;

    inFile.open("example.txt");

    if (inFile) {
        // Способ 1: По словам
        cout << "\na) Чтение по словам:" << endl;
        string word;
        while (inFile >> word) {
            cout << word << " ";
        }
        cout << endl;

        inFile.clear(); // сбрасываем флаги ошибок
        inFile.seekg(0); // переходим в начало файла

        // Способ 2: По символам
        cout << "\nb) Чтение по символам:" << endl;
        char ch;
        while (inFile.get(ch)) {
            cout << ch;
        }

        inFile.close();
    }
}
```

```
void binaryFiles() {
    cout << "\n\n=== РАБОТА С БИНАРНЫМИ ФАЙЛАМИ ===" << endl;
    cout << "===== " << endl;

    // 1. Запись структур в бинарный файл
    cout << "\n1. Запись структур:" << endl;

    struct Person {
        char name[50];
        int age;
        double salary;
    };

    // Создаем данные
    Person people[] = {
        {"Иван Иванов", 30, 50000.50},
        {"Петр Петров", 25, 45000.75},
        {"Анна Сидорова", 35, 60000.00}
    };

    // Записываем в бинарный файл
    ofstream binOut("people.dat", ios::binary);

    if (!binOut) {
        cerr << "Ошибка открытия бинарного файла!" << endl;
        return;
    }

    for (const auto& person : people) {
        binOut.write(reinterpret_cast<const char*>(&person),
            sizeof(Person));
    }

    binOut.close();
    cout << "Записано " << sizeof(people)/sizeof(Person) << " записей" << endl;

    // 2. Чтение из бинарного файла
    cout << "\n2. Чтение структур:" << endl;

    ifstream binIn("people.dat", ios::binary);

    if (!binIn) {
        cerr << "Ошибка открытия файла для чтения!" << endl;
    }
}
```

```

        return;
    }

    Person person;

    cout << left << setw(20) << "Имя"
          << setw(10) << "Возраст"
          << setw(15) << "Зарплата" << endl;
    cout << string(45, '-') << endl;

    while (binIn.read(reinterpret_cast<char*>(&person), sizeof
        cout << left << setw(20) << person.name
          << setw(10) << person.age
          << fixed << setprecision(2) << setw(15) << person
          << endl;
    }

    binIn.close();
}

void fileOperations() {
    cout << "\n\n=== ОПЕРАЦИИ С ФАЙЛАМИ ===" << endl;
    cout << "===== " << endl;

    // 1. Проверка существования файла
    cout << "\n1. Проверка файлов:" << endl;

    ifstream testFile("example.txt");
    if (testFile.good()) {
        cout << "Файл example.txt существует" << endl;
    } else {
        cout << "Файл example.txt не существует" << endl;
    }
    testFile.close();

    // 2. Получение информации о файле
    cout << "\n2. Информация о файле:" << endl;

    ifstream infoFile("example.txt", ios::ate | ios::binary);

    if (infoFile) {
        // Получаем размер файла
        streamsize size = infoFile.tellg();
    }
}

```

```
cout << "Размер файла: " << size << " байт" << endl;

// Перемещаемся в начало
infoFile.seekg(0);

// Считываем весь файл
string content((istreambuf_iterator<char>(infoFile)),
               istreambuf_iterator<char>());

cout << "Первые 100 символов:" << endl;
cout << content.substr(0, 100) << "..." << endl;

infoFile.close();
}

// 3. Добавление в конец файла (append)
cout << "\n3. Добавление в файл:" << endl;

ofstream appendFile("example.txt", ios::app);

if (appendFile) {
    appendFile << "\n=== ДОБАВЛЕННЫЙ ТЕКСТ ===" << endl;
    appendFile << "Эта строка добавлена в конец файла." << endl;
    appendFile.close();
    cout << "Текст добавлен в конец файла" << endl;
}

// 4. Копирование файла
cout << "\n4. Копирование файла:" << endl;

ifstream source("example.txt", ios::binary);
ofstream destination("example_copy.txt", ios::binary);

if (source && destination) {
    destination << source.rdbuf(); // копируем всё содержимое

    cout << "Файл скопирован в example_copy.txt" << endl;

    source.close();
    destination.close();
}

// 5. Обработка ошибок
```

```
cout << "\n5. Обработка ошибок:" << endl;

ifstream errorFile("nonexistent.txt");

if (errorFile.fail()) {
    cerr << "Ошибка: файл не найден!" << endl;
    errorFile.clear(); // очищаем флаги ошибок
}
}

void csvExample() {
    cout << "\n\n=== РАБОТА С CSV ФАЙЛАМИ ===" << endl;
    cout << "===== " << endl;

    // 1. Создание CSV файла
    cout << "\n1. Создание CSV:" << endl;

    ofstream csvOut("data.csv");

    if (csvOut) {
        // Заголовок
        csvOut << "Name, Age, City, Salary" << endl;

        // Данные
        csvOut << "John Doe, 30, New York, 50000" << endl;
        csvOut << "Jane Smith, 25, London, 45000" << endl;
        csvOut << "Bob Johnson, 35, Tokyo, 60000" << endl;
        csvOut << "Alice Brown, 28, Paris, 48000" << endl;

        csvOut.close();
        cout << "CSV файл создан: data.csv" << endl;
    }

    // 2. Чтение и парсинг CSV
    cout << "\n2. Чтение CSV:" << endl;

    ifstream csvIn("data.csv");

    if (csvIn) {
        string line;
        int row = 0;

        cout << left << setw(15) << "Имя"
```

```

        << setw(10) << "Возраст"
        << setw(15) << "Город"
        << setw(10) << "Зарплата" << endl;
    cout << string(50, '-') << endl;

    while (getline(csvIn, line)) {
        if (row == 0) { // пропускаем заголовок
            row++;
            continue;
        }

        // Парсим строку
        stringstream ss(line);
        string name, age, city, salary;

        getline(ss, name, ',');
        getline(ss, age, ',');
        getline(ss, city, ',');
        getline(ss, salary, ',');

        // Выводим с форматированием
        cout << left << setw(15) << name
              << setw(10) << age
              << setw(15) << city
              << setw(10) << salary << endl;

        row++;
    }

    csvIn.close();
}

void bestPractices() {
    cout << "\n\n=== ЛУЧШИЕ ПРАКТИКИ ===" << endl;
    cout << "===== " << endl;

    cout << "1. Всегда проверяйте открытие файла:" << endl;
    cout << "    if (!file) { /* обработка ошибки */ }" << endl;

    cout << "2. Закрывайте файлы явно:" << endl;
    cout << "    file.close();" << endl << endl;
}

```

```

cout << "3. Используйте RAII (Resource Acquisition Is Ini
cout << "    Файл закроется автоматически при выходе из об

cout << "4. Для больших файлов читайте частями:" << endl;
cout << "    while (getline(file, buffer)) { /* обработка

cout << "5. Используйте правильные флаги:" << endl;
cout << "    ios::in - чтение" << endl;
cout << "    ios::out - запись (перезапись)" << endl;
cout << "    ios::app - добавление в конец" << endl;
cout << "    ios::binary - бинарный режим" << endl << endl

cout << "6. Очищайте флаги ошибок:" << endl;
cout << "    file.clear()" << endl << endl;

cout << "7. Для чувствительных данных:" << endl;
cout << "    - Шифруйте данные" << endl;
cout << "    - Проверяйте права доступа" << endl;
cout << "    - Удаляйте временные файлы" << endl;
}

int main() {
    textFiles();
    binaryFiles();
    fileOperations();
    csvExample();
    bestPractices();

    return 0;
}

```

Итог:

Память:

1. **Стек** - для локальных переменных, быстрый, автоматическое управление
2. **Куча** - для динамической памяти, большой размер, ручное управление
3. **Статическая** - для глобальных и static переменных, существует всегда

Указатели:

- `*` для объявления указателя и разыменования
- `&` для получения адреса переменной
- Всегда инициализируйте указатели (`nullptr`)
- Всегда освобождайте динамическую память (`delete` / `delete[]`)

Строки:

- **С-строки:** массивы `char` , опасны, но быстрые
- **`std::string`:** безопасны, много методов, рекомендуются

Файлы:

- **Текстовые:** `ifstream` / `ofstream` , работа с текстом
- **Бинарные:** с флагом `ios::binary` , для любых данных
- Всегда проверяйте успешность открытия
- Закрывайте файлы явно или используйте RAII

Главные опасности:

1. Утечки памяти (не освободили `new`)
2. Переполнение буфера (С-строки)
3. Использование после освобождения
4. Непроверенные операции с файлами

Подробный разбор памяти в C++

1. Основные концепции памяти

1.1 Что такое память в компьютере?

```
#include <iostream>
using namespace std;
```

```
/*
```

ПАМЯТЬ - это последовательность ячеек (байтов), каждая имеет (

Аналогия:

- Память = огромный почтовый ящик с миллионами пронумерованны
- Адрес = номер ящика
- Данные = содержимое ящика
- Указатель = бумажка с номером нужного ящика

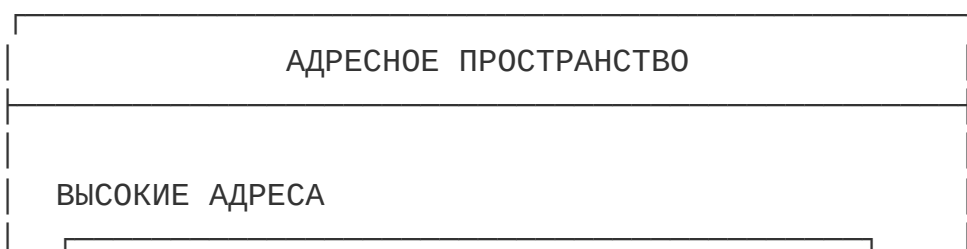
1 байт = 8 бит (может хранить число от 0 до 255 или один символ)
*/

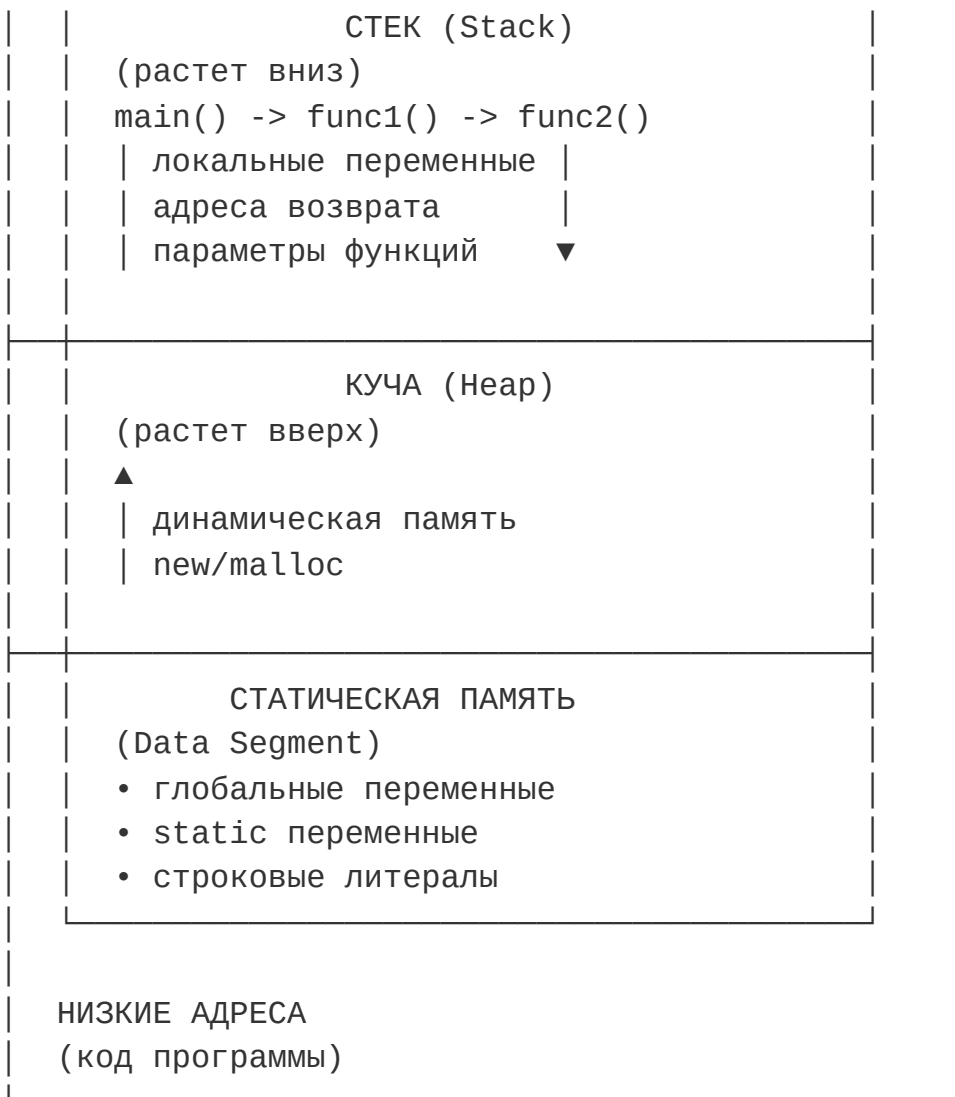
```
void memoryBasics() {
    cout << "=== ОСНОВЫ ПАМЯТИ ===" << endl;

    int x = 42; // int обычно занимает 4 байта
    char c = 'A'; // char занимает 1 байт

    cout << "Размеры типов данных:" << endl;
    cout << "sizeof(char) = " << sizeof(char) << " байт" << endl;
    cout << "sizeof(short) = " << sizeof(short) << " байт" << endl;
    cout << "sizeof(int) = " << sizeof(int) << " байт" << endl;
    cout << "sizeof(long) = " << sizeof(long) << " байт" << endl;
    cout << "sizeof(float) = " << sizeof(float) << " байт" << endl;
    cout << "sizeof(double) = " << sizeof(double) << " байт" << endl;
    cout << "sizeof(bool) = " << sizeof(bool) << " байт" << endl;
}
```

2. Три основных сегмента памяти (диаграмма)





2.1 Детальное сравнение

```
#include <iostream>
using namespace std;

// Глобальные переменные - статическая память
int global_var = 100;          // инициализированные данные
int uninitialized_global;     // неинициализированные (BSS)

void compareMemorySegments() {
    cout << "\n=== СРАВНЕНИЕ СЕГМЕНТОВ ПАМЯТИ ===" << endl;
    cout << "===== " << endl;

    cout << "\n1. СТЕК (Stack):" << endl;
    cout << "    • Локальные переменные функций" << endl;
```

```

cout << "    • Параметры функций" << endl;
cout << "    • Адреса возврата" << endl;
cout << "    • Автоматическое управление" << endl;
cout << "    • Быстрый доступ" << endl;
cout << "    • Ограниченный размер (1-8 МБ)" << endl;
cout << "    • LIFO (последний пришел - первый вышел)" << endl;

cout << "\n2. КУЧА (Heap):" << endl;
cout << "    • Динамическая память (new/delete)" << endl;
cout << "    • Большой размер (ограничен ОЗУ)" << endl;
cout << "    • Ручное управление" << endl;
cout << "    • Медленнее стека" << endl;
cout << "    • Можно выделять/освобождать в любом порядке"

cout << "\n3. СТАТИЧЕСКАЯ ПАМЯТЬ:" << endl;
cout << "    • Глобальные переменные" << endl;
cout << "    • static локальные переменные" << endl;
cout << "    • Строковые литералы" << endl;
cout << "    • Существует всю программу" << endl;
cout << "    • Инициализируется при запуске" << endl;

cout << "\n4. КОД (Code/Text):" << endl;
cout << "    • Исполняемые инструкции" << endl;
cout << "    • Только для чтения" << endl;
cout << "    • Находится в низких адресах" << endl;
}

```

3. СТЕК в деталях

3.1 Как работает стек

КЛЮЧЕВАЯ ИДЕЯ: Стек работает как стопка тарелок!

ДОБАВЛЕНИЕ (PUSH):

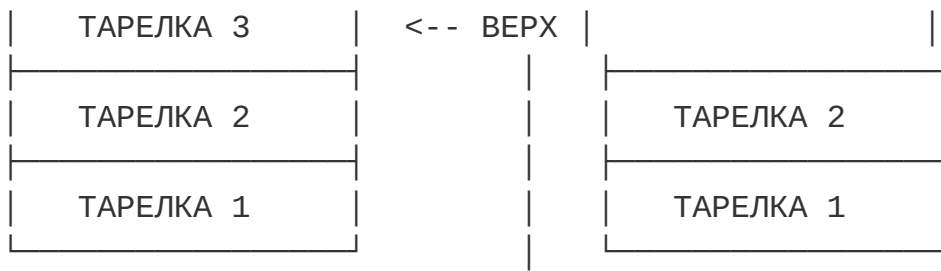
НОВАЯ
ТАРЕЛКА



УДАЛЕНИЕ (POP):

ВЕРХНЯЯ
ТАРЕЛКА





```
#include <iostream>
using namespace std;

void functionC() {
    cout << " Входим в functionC()" << endl;
    int c_var = 30; // Локальная переменная в стеке
    cout << " c_var адрес: " << &c_var << endl;
    cout << " Выходим из functionC()" << endl;
    // c_var автоматически уничтожается здесь
}

void functionB() {
    cout << "Входим в functionB()" << endl;
    int b_var = 20; // Локальная переменная в стеке
    cout << "b_var адрес: " << &b_var << endl;

    functionC(); // Вызов другой функции

    cout << "Выходим из functionB()" << endl;
    // b_var автоматически уничтожается здесь
}

void stackDemo() {
    cout << "\n=== ДЕМОНСТРАЦИЯ СТЕКА ===" << endl;
    cout << "===== " << endl;

    cout << "Входим в stackDemo()" << endl;
    int a_var = 10; // Локальная переменная в стеке
    cout << "a_var адрес: " << &a_var << endl;

    functionB(); // Вызов функции

    cout << "Выходим из stackDemo()" << endl;
    // a_var автоматически уничтожается здесь
}
```

```
// Визуализация стека во время выполнения:
cout << "\nДИАГРАММА СТЕКА во время functionC():" << endl;
cout << "
|         functionC()         |" << endl;
cout << "    - c_var = 30              |" << endl;
cout << "    - адрес возврата         |" << endl;
cout << "
|         functionB()         |" << endl;
cout << "    - b_var = 20              |" << endl;
cout << "    - адрес возврата         |" << endl;
cout << "
|         stackDemo()         |" << endl;
cout << "    - a_var = 10              |" << endl;
cout << "    - адрес возврата         |" << endl;
cout << "
|         main()              |" << endl;
cout << "
|                             |" << endl;
}
```

// Пример переполнения стека

```
void recursiveFunction(int depth) {
    int array[1000]; // Выделяем 4KB в стеке (1000 * 4 байта)

    cout << "Рекурсия глубина: " << depth
          << ", адрес array: " << &array << endl;

    if (depth < 100) { // Уменьшим для безопасности
        recursiveFunction(depth + 1);
    }
}
```

```
void stackOverflowDemo() {
    cout << "\n=== ПЕРЕПОЛНЕНИЕ СТЕКА (Stack Overflow) ===" << endl;
    cout << "===== " << endl;

    try {
        recursiveFunction(1);
    } catch (...) {
        cout << "Произошло переполнение стека!" << endl;
    }

    cout << "\nПочему происходит переполнение:" << endl;
    cout << "1. Каждый вызов функции выделяет память в стеке"
```

```

cout << "2. При глубокой рекурсии память накапливается" << endl;
cout << "3. Когда стек достигает предела - КРАХ!" << endl;
cout << "\nРешение: используйте итерацию или выделяйте па
}

```

3.2 Память для массивов в стеке

```

void stackArrays() {
    cout << "\n=== МАССИВЫ В СТЕКЕ ===" << endl;
    cout << "===== " << endl;

    int smallArray[5] = {1, 2, 3, 4, 5}; // 20 байт в стеке

    cout << "Массив в стеке:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << "smallArray[" << i << "] адрес: "
            << &smallArray[i] << ", значение: " << smallArray[i] << endl;
    }

    cout << "\nМассив расположен в памяти непрерывно:" << endl;
    cout << "smallArray[0]: " << &smallArray[0] << endl;
    cout << "smallArray[1]: " << &smallArray[1]
        << " (разница: " << (char*)&smallArray[1] - (char*)&smallArray[0] << endl;
    cout << "smallArray[2]: " << &smallArray[2]
        << " (разница: " << (char*)&smallArray[2] - (char*)&smallArray[0] << endl;

    // ОПАСНОСТЬ: большой массив в стеке
    // int hugeArray[1000000]; // 4МБ - скорее всего переполнит

    cout << "\nБезопасный предел для стека: несколько мегабайт" << endl;
    cout << "Для больших данных используйте кучу!" << endl;
}

```

4. КУЧА в деталях

4.1 Как работает куча

КЛЮЧЕВАЯ ИДЕЯ: Куча - это склад с полками, где вы сами ищете



```
#include <iostream>
using namespace std;

void heapDemo() {
    cout << "\n=== ДЕМОНСТРАЦИЯ КУЧИ ===" << endl;
    cout << "===== " << endl;

    // 1. Выделение памяти в куче
    cout << "1. Выделение памяти (new):" << endl;
    int* single = new int(42);
    cout << "    Выделен int в куче: " << *single << endl;
    cout << "    Адрес в куче: " << single << endl;
    cout << "    Адрес указателя в стеке: " << &single << endl;

    // 2. Массивы в куче
    cout << "\n2. Массивы в куче:" << endl;
    int size = 5;
    int* array = new int[size]; // массив в куче

    for (int i = 0; i < size; i++) {
        array[i] = i * 10;
    }
}
```

```

        cout << "    array[" << i << "] = " << array[i]
            << ", адрес: " << &array[i] << endl;
    }

// 3. Большие объекты в куче
cout << "\n3. Большие объекты (безопасно в куче):" << endl;
int bigSize = 1000000; // 1 миллион элементов = 4MB
int* bigArray = new int[bigSize];
cout << "    Выделено " << bigSize << " элементов ("
    << bigSize * sizeof(int) << " байт)" << endl;

// 4. ОСВОБОЖДЕНИЕ памяти - ВАЖНО!
cout << "\n4. Освобождение памяти (delete):" << endl;
delete single; // для одиночных объектов
delete[] array; // для массивов с []
delete[] bigArray; // для больших массивов

cout << "    Память освобождена" << endl;

// 5. Опасности
cout << "\n5. ОПАСНОСТИ КУЧИ:" << endl;
int* danger = new int(100);

// УТЕЧКА ПАМЯТИ (Memory Leak)
// Если не сделать delete danger - память никогда не освободится

// ДВОЙНОЕ УДАЛЕНИЕ (Double Free)
delete danger;
// delete danger; // КАТАСТРОФА! Повторное удаление

// ДОСТУП ПОСЛЕ УДАЛЕНИЯ (Use After Free)
// *danger = 200; // ОПАСНО! Обращение к освобожденной памяти

danger = nullptr; // Хорошая практика после delete

// 6. Умные указатели (решение проблем)
cout << "\n6. Решение: умные указатели" << endl;
cout << "    unique_ptr - автоматически удаляет объект" << endl;
cout << "    shared_ptr - считает ссылки" << endl;
}

```

4.2 Процесс выделения памяти в куче

```

void heapAllocationProcess() {
    cout << "\n=== ПРОЦЕСС ВЫДЕЛЕНИЯ ПАМЯТИ В КУЧЕ ===" << endl;
    cout << "===== " << endl;

    cout << "Когда вы пишете: int* ptr = new int(42);" << endl;
    cout << "Происходит следующее:" << endl;
    cout << endl;

    cout << "ШАГ 1: Запрос к менеджеру памяти" << endl;
    cout << "        Программа: 'Дай мне 4 байта для int'" << endl;
    cout << endl;

    cout << "ШАГ 2: Поиск свободного блока" << endl;
    cout << "        Менеджер памяти ищет в куче свободный блок" << endl;
    cout << endl;

    cout << "ШАГ 3: Разметка блока" << endl;
    cout << "        |————— БЛОК ПАМЯТИ —————|" << endl;
    cout << "        | МЕТАДАННЫЕ | ДАННЫЕ (4 байта) |" << endl;
    cout << "        | (8-16 байт) |" << endl;
    cout << "        |—————|" << endl;
    cout << "        Метаданные: размер, флаги, ссылки..." << endl;
    cout << endl;

    cout << "ШАГ 4: Инициализация" << endl;
    cout << "        В выделенные 4 байта записывается значение" << endl;
    cout << endl;

    cout << "ШАГ 5: Возврат указателя" << endl;
    cout << "        Возвращается адрес начала блока данных (начало int)" << endl;
    cout << endl;

    cout << "ПРИМЕЧАНИЕ: Метаданные хранятся скрыто от программы" << endl;
    cout << "но занимают дополнительную память (оверхед)." << endl;
    cout << "Вот почему мелкие объекты лучше хранить в стеке!" << endl;
}

```

5. СТАТИЧЕСКАЯ ПАМЯТЬ

5.1 Глобальные и static переменные

```
#include <iostream>
using namespace std;

// Глобальные переменные - в статической памяти
int global_counter = 0;           // Инициализированная
const int MAX_SIZE = 100;        // Константная
int uninitialized_global;        // Неинициализированная (BSF

void staticMemoryDemo() {
    cout << "\n=== СТАТИЧЕСКАЯ ПАМЯТЬ ===" << endl;
    cout << "===== " << endl;

    // static локальная переменная
    static int function_call_count = 0;
    function_call_count++;

    cout << "Функция вызвана " << function_call_count << " раз" << endl;

    // Еще локальная static переменная
    static char* message = "Hello from static memory";
    cout << "Сообщение: " << message << endl;
    cout << "Адрес сообщения: " << (void*)message << endl;

    // Строковые литералы тоже в статической памяти
    char* literal = "String literal";
    cout << "Литерал: " << literal << endl;
    cout << "Адрес литерала: " << (void*)literal << endl;

    cout << "\nОСОБЕННОСТИ STATIC переменных:" << endl;
    cout << "1. Инициализируются один раз при первом вызове" << endl;
    cout << "2. Сохраняют значение между вызовами функции" << endl;
    cout << "3. Существуют всю программу" << endl;
    cout << "4. Видимы только в своей области видимости" << endl;
}

// Еще пример с static
void counterFunction() {
    static int counter = 0; // Инициализируется только один раз
    int temp = 0;          // Создается заново каждый вызов
```

```

    counter++;
    temp++;

    cout << "counter = " << counter << ", temp = " << temp <<
}

```

5.2 Схема статической памяти

```

void staticMemoryLayout() {
    cout << "\n=== СТРУКТУРА СТАТИЧЕСКОЙ ПАМЯТИ ===" << endl;
    cout << "===== " << endl;

    cout << "Статическая память состоит из сегментов:" << endl;
    cout << endl;

    cout << " " << endl;
    cout << " |          СТАТИЧЕСКАЯ ПАМЯТЬ          | " << endl;
    cout << " |-----| " << endl;
    cout << " | .data (инициализированные данные) | " << endl;
    cout << " | • int x = 42;                        | " << endl;
    cout << " | • const double PI = 3.14;           | " << endl;
    cout << " |-----| " << endl;
    cout << " | .bss (неинициализированные)        | " << endl;
    cout << " | • int y;                            | " << endl;
    cout << " | • static int z;                     | " << endl;
    cout << " |-----| " << endl;
    cout << " | .rodata (только для чтения)         | " << endl;
    cout << " | • строковые литералы                | " << endl;
    cout << " | • const глобальные константы        | " << endl;
    cout << " |-----| " << endl;

    cout << "\nПримеры:" << endl;

    // .data сегмент
    static int initialized_static = 100;
    cout << "initialized_static = " << initialized_static
        << " (в .data)" << endl;

    // .bss сегмент
    static int uninitialized_static;
    cout << "uninitialized_static = " << uninitialized_static

```

```

    << " (в .bss, инициализирован 0)" << endl;

    // .rodata сегмент
    const char* hello = "Hello, World!";
    cout << "hello = \"\" << hello << "\" (в .rodata)" << endl
}

```

6. ПОЛНАЯ ДИАГРАММА ПАМЯТИ ПРОГРАММЫ

```

#include <iostream>
#include <vector>
using namespace std;

// Глобальные переменные
int global_data = 100;           // .data
int global_bss;                 // .bss
const char* global_rodata = "Constant"; // .rodata

void completeMemoryDiagram() {
    cout << "\n=== ПОЛНАЯ ДИАГРАММА ПАМЯТИ ===" << endl;
    cout << "===== " << endl;

    // Локальные переменные (стек)
    int stack_var = 10;
    double stack_array[3] = {1.1, 2.2, 3.3};

    // Динамическая память (куча)
    int* heap_single = new int(777);
    vector<int>* heap_vector = new vector<int>{1, 2, 3, 4, 5}

    // Статические (в функции)
    static int static_local = 999;

    cout << "\n|_____
    cout << "|                ПАМЯТЬ ПРОГРАММЫ
    cout << "|_____
    cout << "|
    cout << "|    ВЫСОКИЕ АДРЕСА
    cout << "|    |_____
    cout << "|    |                СТЕК
    cout << "|    |    • stack_var = " << stack_var << "

```

```

cout << " | | • stack_array[3]
cout << " | | • Адреса возврата
cout << " | | • Параметры функций
cout << " | |
cout << " | | _____
cout << " | | КУЧА
cout << " | | ▲
cout << " | | | • heap_single = " << *heap_single << "
cout << " | | | • heap_vector
cout << " | | | • Динамические массивы
cout << " | |
cout << " | | _____
cout << " | | СТАТИЧЕСКАЯ ПАМЯТЬ
cout << " | | • global_data = " << global_data << "
cout << " | | • global_bss = " << global_bss << "
cout << " | | • static_local = " << static_local << "
cout << " | | • Строковые литералы
cout << " | |
cout << " | | _____
cout << " | | НИЗКИЕ АДРЕСА
cout << " | | _____
cout << " | | КОД
cout << " | | • Инструкции программы
cout << " | | • Только для чтения
cout << " | | _____
cout << " | | _____
cout << " | | _____

```

```

// Освобождаем динамическую память
delete heap_single;
delete heap_vector;

```

```

cout << "\nОСНОВНЫЕ ВЫВОДЫ:" << endl;
cout << "1. Код - только для чтения, в низких адресах" <<
cout << "2. Статическая память - данные на всю программу"
cout << "3. Куча - растет вверх, динамическое выделение"
cout << "4. Стек - растет вниз, автоматическое управление"
}

```

7. ПРАКТИЧЕСКИЕ ПРИМЕРЫ И ОШИБКИ

7.1 Распространенные ошибки с памятью

```

void commonMemoryErrors() {
    cout << "\n=== РАСПРОСТРАНЕННЫЕ ОШИБКИ ===" << endl;
    cout << "===== " << endl;

    cout << "\n1. УТЕЧКА ПАМЯТИ (Memory Leak):" << endl;
    cout << "    int* ptr = new int(100);" << endl;
    cout << "    // Забыли delete ptr - память никогда не освободится" << endl;
    cout << "    Решение: Всегда парные new/delete" << endl;

    cout << "\n2. ДОСТУП ПОСЛЕ УДАЛЕНИЯ (Use After Free):" << endl;
    cout << "    int* ptr = new int(100);" << endl;
    cout << "    delete ptr;" << endl;
    cout << "    *ptr = 200; // КАТАСТРОФА! Обращение к освобожденной памяти" << endl;
    cout << "    Решение: После delete устанавливайте ptr = nullptr" << endl;

    cout << "\n3. ДВОЙНОЕ УДАЛЕНИЕ (Double Free):" << endl;
    cout << "    int* ptr = new int(100);" << endl;
    cout << "    delete ptr;" << endl;
    cout << "    delete ptr; // ВТОРОЙ РАЗ - ОШИБКА!" << endl;
    cout << "    Решение: Только один delete на один new" << endl;

    cout << "\n4. ВИСЯЧИЙ УКАЗАТЕЛЬ (Dangling Pointer):" << endl;
    cout << "    int* ptr = new int(100);" << endl;
    cout << "    int* ptr2 = ptr; // оба указывают на одно место в памяти" << endl;
    cout << "    delete ptr;      // освободили память" << endl;
    cout << "    *ptr2 = 200;      // ptr2 теперь висячий указатель" << endl;

    cout << "\n5. ПЕРЕПОЛНЕНИЕ СТЕКА (Stack Overflow):" << endl;
    cout << "    void recursive() { int big[10000]; recursive();" << endl;
    cout << "    Решение: Для больших данных используйте кучу" << endl;

    cout << "\n6. ПЕРЕПОЛНЕНИЕ БУФЕРА (Buffer Overflow):" << endl;
    cout << "    char buffer[10];" << endl;
    cout << "    strcpy(buffer, \"Очень длинная строка\"); // переполнение" << endl;
    cout << "    Решение: Проверяйте границы, используйте безопасные функции" << endl;
}

// Примеры ошибок
void demonstrateErrors() {
    cout << "\n--- Примеры ошибок на практике ---" << endl;

```

```

// 1. Утечка памяти
for (int i = 0; i < 1000; i++) {
    int* leak = new int(i); // Выделяем...
    // Забыли delete - 1000 утечек!
}
cout << "Создано 1000 утечек памяти!" << endl;

// 2. Висячий указатель
int* original = new int(42);
int* copy = original; // Оба указывают на одно место

delete original;      // Освободили память
original = nullptr;   // Хорошо

// А вот copy теперь висячий указатель!
// *copy = 100; // ОПАСНО - непредсказуемое поведение

cout << "Создан висячий указатель" << endl;
}

```

7.2 Правильные практики

```

void bestPractices() {
    cout << "\n=== ЛУЧШИЕ ПРАКТИКИ РАБОТЫ С ПАМЯТЬЮ ===" << endl;
    cout << "===== " << endl;

    cout << "\n1. ИСПОЛЬЗУЙТЕ СТЕК, когда можно:" << endl;
    cout << "    • Для локальных переменных" << endl;
    cout << "    • Для небольших массивов" << endl;
    cout << "    • Для временных объектов" << endl;

    cout << "\n2. ИСПОЛЬЗУЙТЕ КУЧУ, когда нужно:" << endl;
    cout << "    • Для больших объектов" << endl;
    cout << "    • Когда размер известен только во время выполнения" << endl;
    cout << "    • Для данных, которые должны жить дольше функции" << endl;

    cout << "\n3. ЗОЛОТЫЕ ПРАВИЛА динамической памяти:" << endl;
    cout << "    1. Каждому new соответствует delete" << endl;
    cout << "    2. Каждому new[] соответствует delete[]" << endl;
    cout << "    3. После delete устанавливайте указатель в nullptr" << endl;
}

```

```

cout << "    4. Проверяйте указатель перед использованием"

cout << "\n4. ИСПОЛЬЗУЙТЕ УМНЫЕ УКАЗАТЕЛИ:" << endl;
cout << "    • unique_ptr - для исключительного владения" << endl;
cout << "    • shared_ptr - для разделяемого владения" << endl;
cout << "    • weak_ptr - для наблюдения без владения" << endl;

cout << "\n5. ПРАВИЛО RAII (Resource Acquisition Is Initialization)" << endl;
cout << "    • Получайте ресурсы в конструкторе" << endl;
cout << "    • Освобождайте ресурсы в деструкторе" << endl;
cout << "    • Ресурсы освобождаются автоматически" << endl;

cout << "\n6. ИЗБЕГАЙТЕ РУЧНОГО УПРАВЛЕНИЯ ПАМЯТЬЮ:" << endl;
cout << "    • Используйте vector вместо new[]/delete[]" << endl;
cout << "    • Используйте string вместо char[]" << endl;
cout << "    • Используйте умные указатели вместо raw указателей" << endl;
}

// Пример правильного кода
void goodCodeExample() {
    cout << "\n=== ПРИМЕР ПРАВИЛЬНОГО КОДА ===" << endl;
    cout << "===== " << endl;

    // 1. Стек для мелких данных
    int local_variable = 42;           // ОК
    int small_array[10];               // ОК (если небольшой)

    // 2. Куча для больших данных с RAII
    vector<int> big_data(1000000);     // ЛУЧШЕ чем new[]/delete[]

    // 3. Умные указатели
    unique_ptr<int> smart_ptr = make_unique<int>(100);
    // Автоматически удалится при выходе из области видимости

    // 4. Строки вместо char[]
    string text = "Hello";             // ЛУЧШЕ чем char text[]

    cout << "Этот код безопасен и не имеет утечек памяти!" << endl;
}

```

8. ИТОГОВАЯ ПРОГРАММА-ПРИМЕР

```

#include <iostream>
#include <memory>
#include <vector>
using namespace std;

// Глобальная переменная - статическая память
static int program_runs = 0;

class MemoryDemo {
private:
    int id;
    int* heap_data;  // Динамические данные

public:
    MemoryDemo(int id_val) : id(id_val) {
        heap_data = new int[100];  // Выделяем в куче
        cout << "Создан объект " << id << " (память в куче)" << endl;
    }

    ~MemoryDemo() {
        delete[] heap_data;  // ОСВОБОЖДАЕМ в деструкторе
        cout << "Уничтожен объект " << id << " (память освобождена)" << endl;
    }

    // Правило трех: если есть деструктор, нужны также
    // конструктор копирования и оператор присваивания
    MemoryDemo(const MemoryDemo& other) = delete;  // запрещаем копирование
    MemoryDemo& operator=(const MemoryDemo& other) = delete;

};

void comprehensiveExample() {
    cout << "\n\n=== КОМПЛЕКСНЫЙ ПРИМЕР ===" << endl;
    cout << "===== " << endl;

    program_runs++;
    cout << "Запуск программы #" << program_runs << endl;

    // 1. Данные в стеке
    cout << "\n1. ДАННЫЕ В СТЕКЕ:" << endl;
    int stack_int = 42;

```

```

double stack_array[3] = {1.1, 2.2, 3.3};

cout << "    stack_int = " << stack_int << endl;
cout << "    Адрес stack_int: " << &stack_int << endl;

// 2. Данные в куче (старый способ - ОПАСНО)
cout << "\n2. ДАННЫЕ В КУЧЕ (опасный способ):" << endl;
int* raw_ptr = new int(100);
cout << "    *raw_ptr = " << *raw_ptr << endl;
delete raw_ptr; // Не забыть!
raw_ptr = nullptr;

// 3. Данные в куче (безопасный способ)
cout << "\n3. ДАННЫЕ В КУЧЕ (безопасный способ):" << endl;
unique_ptr<int> smart_ptr = make_unique<int>(200);
cout << "    *smart_ptr = " << *smart_ptr << endl;
// Автоматически удалится

// 4. RAII объект
cout << "\n4. RAII ОБЪЕКТ:" << endl;
{
    MemoryDemo demo(1); // Конструктор выделяет память
    // При выходе из блока деструктор освободит память
}

// 5. Вектор (лучше чем динамический массив)
cout << "\n5. ВЕКТОР (альтернатива new[]):" << endl;
vector<int> numbers = {1, 2, 3, 4, 5};
numbers.push_back(6); // Автоматически управляет памятью

cout << "    Вектор содержит: ";
for (int n : numbers) {
    cout << n << " ";
}
cout << endl;

// 6. Рекурсия и стек
cout << "\n6. РЕКУРСИЯ И СТЕК:" << endl;
cout << "    Каждый вызов функции добавляет кадр в стек" << endl;
cout << "    Глубокая рекурсия = переполнение стека" << endl;

// 7. Статические данные
cout << "\n7. СТАТИЧЕСКИЕ ДАННЫЕ:" << endl;

```

```

static int call_count = 0;
call_count++;
cout << "    Функция вызвана " << call_count << " раз(a)" << endl;
}

int main() {
    cout << "
    cout << "||          ГЛУБОКИЙ РАЗБОР ПАМЯТИ В C++          ||"
    cout << "||          для начинающих          ||"
    cout << "
    memoryBasics();
    compareMemorySegments();

    stackDemo();
    stackArrays();

    heapDemo();
    heapAllocationProcess();

    staticMemoryDemo();
    staticMemoryLayout();

    completeMemoryDiagram();

    commonMemoryErrors();
    demonstrateErrors();

    bestPractices();
    goodCodeExample();

    comprehensiveExample();

    cout << "\n\n★ ВАЖНЫЕ ВЫВОДЫ ★" << endl;
    cout << "—————" << endl;
    cout << "1. СТЕК: быстрый, автоматический, маленький" << endl;
    cout << "2. КУЧА: медленный, ручное управление, большой" << endl;
    cout << "3. СТАТИЧЕСКАЯ: на всю программу, инициализация" << endl;
    cout << "4. Всегда освобождайте динамическую память!" << endl;
    cout << "5. Используйте умные указатели и контейнеры STL" << endl;
}

```

```
    return 0;  
}
```

Ключевые диаграммы для запоминания:

Диаграмма 1: Сравнение стек vs куча

СТЕК	КУЧА
Автоматическое	Ручное управление
Быстро	Медленно
Маленький	Большой
LIFO	Любой порядок
Локальные переменные	Динамические объекты

Диаграмма 2: Жизненный цикл

ВРЕМЯ ЖИЗНИ		
СТЕК	КУЧА	СТАТИЧЕСКАЯ
Функция вызвана	До delete	Вся программа
Функция завершена	delete вызван	Завершение программы

Диаграмма 3: Когда что использовать



